Encryption in Transit

private is not the same as safe

Watch the 5-min video

In one line: TLS gives confidentiality, integrity and server authentication (plus client authentication with mTLS); use it on every hop, including inside the private network, with TLS 1.2+ only, HSTS for browsers, and automated certificates.

1 Plain text behind the load balancer

The public site uses HTTPS, but the load balancer terminates TLS and forwards plain HTTP to the services: “it's a private network”. Then an attacker lands on one machine, or a misconfigured traffic mirror copies packets, and every password, token and customer record is readable. Encrypting only at the edge protects the easy part.

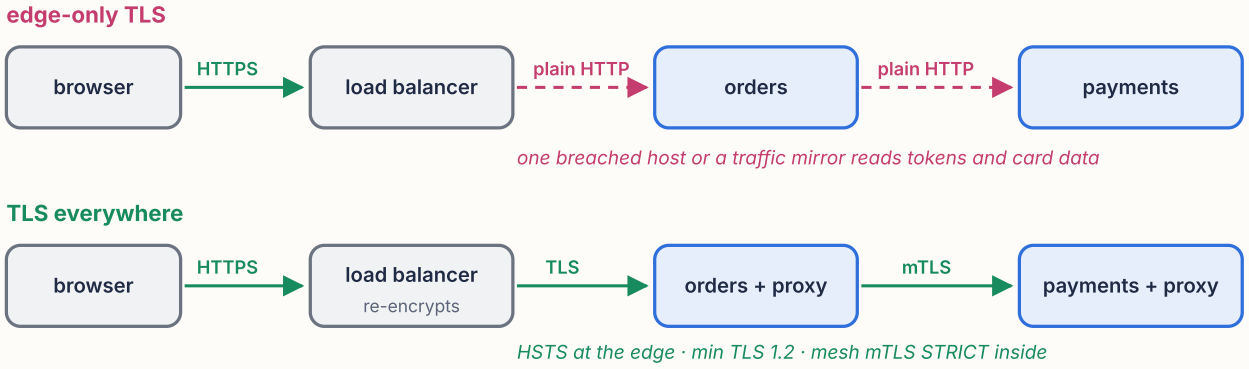


Figure 1. Terminating TLS at the edge leaves every internal hop exposed. Re-encrypt at the load balancer and use mesh mTLS between services.

2 What TLS gives you

- **Confidentiality**: symmetric encryption (AES-GCM or ChaCha20-Poly1305) of every record.
- **Integrity**: the AEAD tag detects any change in transit.
- **Server authentication**: a certificate chain to a trusted CA, plus a signature with the server's private key; the client checks the hostname.
- **Client authentication** only with mutual TLS (lesson 45).

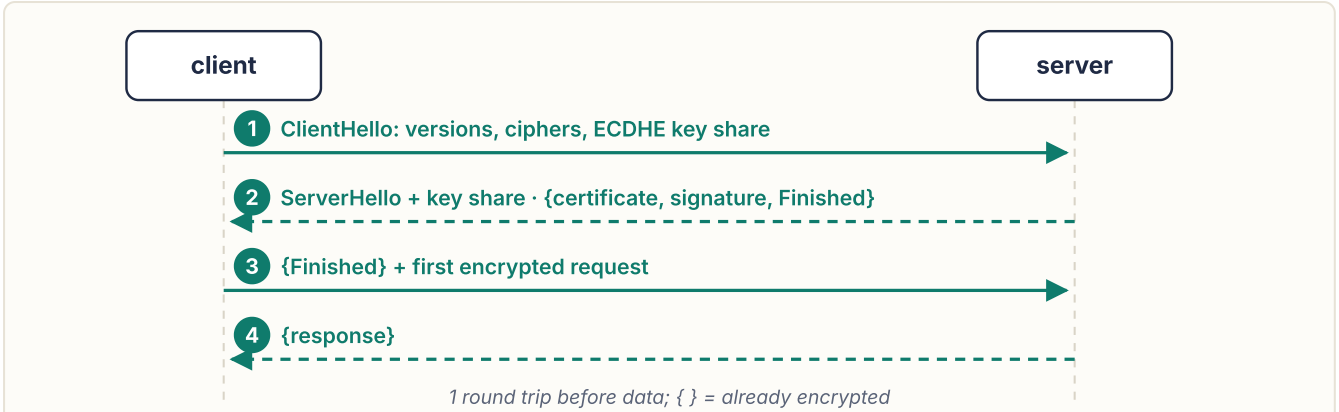


Figure 2. TLS 1.3 (RFC 8446) needs one round trip. The cost is mostly the handshake, so reuse connections (keep-alive, HTTP/2, pools).

3 Why encrypt internal traffic?

Private is not safe. A breached host can sniff its neighbours. Cloud networks are shared infrastructure. Insiders, debug proxies and packet captures see raw traffic. Compliance regimes (PCI DSS for card data, HIPAA for health data) require or strongly expect encryption in transit. The modern default is **TLS on every hop**, which zero trust assumes anyway (lesson 53).

4 HSTS for browsers

`Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` tells the browser to use only HTTPS for this site for a year. That stops **SSL stripping**, where an attacker on public Wi-Fi intercepts the first plain-HTTP request and keeps the victim on HTTP. The first visit is still exposed unless the domain is on the browsers' **preload list** (hstspreload.org), which requires exactly these three directives.

5 Certificates, protocols and ciphers

TLS everywhere means certificates everywhere, and an expired certificate is a classic outage. Automate: ACME with Let's Encrypt (90-day certificates), cert-manager in Kubernetes, a mesh CA for short-lived internal certificates. Monitor expiry and alert weeks ahead. Public certificate lifetimes are shrinking (CA/Browser Forum: 200 days from March 2026, 47 days by 2029), so manual renewal no longer works. Keep private keys out of code and images.

Protocol	Status	Why
SSL 3.0, TLS 1.0, TLS 1.1	disable	weak constructions (POODLE, BEAST); TLS 1.0/1.1 formally deprecated by RFC 8996 (2021)
TLS 1.2	allow, with ECDHE + AES-GCM / ChaCha20	still needed by older clients; avoid static-RSA key exchange and CBC suites
TLS 1.3	prefer	only five AEAD suites, forward secrecy always, 1-RTT handshake

Forward secrecy (ephemeral ECDHE keys) means a server key stolen later can't decrypt traffic recorded earlier. TLS 1.3 makes it mandatory. Beware 0-RTT resumption in TLS 1.3: early data can be replayed, so allow it only for idempotent requests.

WORKED EXAMPLE: TLS 1.2+ ON ALL INTERNAL ENDPOINTS

The team sets a minimum of TLS 1.2 on the load balancer and API gateway (e.g. an AWS ALB security policy or Azure App Gateway SSL policy), and Istio `PeerAuthentication` mode `STRICT` for pod-to-pod traffic. An admission policy (OPA Gatekeeper or Kyverno) rejects any Service or Ingress exposing plain HTTP. A weekly scan (testssl.sh, sslyze) flags endpoints that still accept TLS 1.0/1.1 or weak ciphers, and expiry alerts fire 30 days ahead.

COMMON MISTAKES

- TLS at the edge only; plain HTTP behind the load balancer.
- Disabling certificate verification in internal clients (`verify=False`), which removes authentication entirely.
- Certificates renewed by hand and a calendar reminder.
- HSTS with a long max-age and `includeSubDomains` before every subdomain supports HTTPS.

6 Key terms

TLS: protocol for encrypted, authenticated connections

Forward secrecy: past sessions stay safe if the long-term key leaks

TLS termination: where TLS is decrypted, e.g. at the load balancer

AEAD: cipher mode that encrypts and authenticates, e.g. AES-GCM

HSTS: header forcing HTTPS for a domain

ACME: protocol for automated certificate issuance

7 Interview questions

Q1. What does TLS give you? BASIC

Confidentiality, integrity and server authentication, and with mutual TLS, client authentication too.

Q2. What is HSTS and what attack does it stop? BASIC

A response header telling browsers to use only HTTPS for the domain for max-age seconds; it stops SSL-stripping downgrades after the first visit (or from the first, if preloaded).

Q3. Why encrypt traffic inside a private network? SCENARIO

Private doesn't mean safe: a breached host, shared cloud infrastructure, insiders or misconfigured logging can expose plain traffic, and compliance often requires it.

Q4. Production went down because a certificate expired. How do you prevent a repeat? SCENARIO

Automate issuance and renewal (ACME, cert-manager, mesh CA), monitor expiry for every endpoint with alerts weeks ahead, and inventory where certificates live.

Q5. What is forward secrecy, and why does it matter? TRICKY

Each session uses ephemeral (EC)DHE keys, so stealing the server's private key later can't decrypt recorded traffic. TLS 1.3 requires it.

Q6. Isn't TLS everywhere too slow? TRICKY

On modern CPUs with AES instructions, record encryption is cheap; the handshake is the main cost, and connection reuse, session resumption and TLS 1.3's 1-RTT handshake make it small.

Next: SD-M-009 · Encryption at Rest and KMS backendroadmap.com